OmniStack: AI-Integrated Automated Deployment Platform for Private Servers

(Project Proposal)

Project Code
OmniStack

Project Advisor
Mr. Amir Zia (internal)

Project Manager
Dr. Muhammad Ilyas

Project Team
Talha Saleem — BSCS51F23S012 — Team Lead / AI & ML
Salman Khalid — BSCS51F23S045 — Team Member / DevOps & Platform

Submission Date
October 13, 2026

Table of Contents

1. Abstract....2
2. Background and Justification....2
3. Project Methodology....3
4. Project Scope....3
5. High level Project Plan....4
6. References....4

1. Abstract

Private, self-hosted servers are increasingly used by small teams and students to run their own applications instead of relying on costly, vendor-locked cloud platforms. However, deploying and maintaining applications on self-hosted infrastructure is largely manual: existing open-source deployment tools (e.g., Coolify) automate container deployment but provide little intelligence about the health of containers once they are running. Administrators must manually watch dashboards to notice early warning signs, such as rising memory usage, CPU spikes and crash loops, before a service actually fails. Most monitoring tools also use fixed rules like "alert if CPU is above 80%", which give false alarms during normal busy times and miss slow problems like a small memory leak.

This project proposes OmniStack, an AI-Integrated Automated Deployment Platform for private servers. It combines (a) a deployment engine that takes an application from GitHub, builds it with Docker and runs it on a self-hosted server, and (b) an AI/ML-based Container Health Anomaly Detector that continuously checks the resource metrics of each container (CPU, memory, restart count, network I/O) and uses an unsupervised machine learning model (Isolation Forest) to flag containers whose behavior is different from normal operation. The results are shown to the user on a simple web dashboard.

The objective is to deliver a working, evaluated anomaly-detection model integrated with a functioning private-server deployment pipeline, demonstrating that container health can be monitored intelligently without a third-party cloud provider. To achieve this we will take the following steps: (1) learn the required tools (Docker, Linux, Git, scikit-learn); (2) build the deployment engine and the dashboard; (3) collect normal container metrics and create abnormal data by injecting faults such as memory leaks, CPU spikes and crash loops; (4) train and test the Isolation Forest model and compare it with a simple fixed-threshold rule; (5) connect the model to the platform so the health status is visible on the dashboard; and (6) test the full system with sample applications.

This benefits small teams and students who self-host applications by giving earlier, more reliable warning of failing services, and gives the project team hands-on, defensible experience applying classical machine learning to a real infrastructure problem.

2. Background and Justification

2.1 Why This Area and Problem Matter

Today most applications run in containers (Docker). Cloud platforms like Heroku, Vercel and AWS make deployment easy, but they charge monthly fees, and this is hard for students and small teams. Many of them rent or own a private server instead. The problem is that running apps on a private server is mostly manual work, and nobody tells you that a container is slowly getting sick. A memory leak can eat the RAM for days and then the whole server crashes. Simple uptime checks only tell us after the service is already down.

2.2 Existing Solutions

Table 1 shows the tools people use now. Coolify [5], Portainer [6] and Dokku [7] help to deploy and manage containers on private servers. Prometheus with Alertmanager [8] is a popular monitoring tool. All of them depend on fixed alert rules or on a person watching the graphs.

Feature	Coolify	Portainer / Dokku	Prometheus + Alertmanager	OmniStack (proposed)
Deploy from GitHub	Yes	Dokku: yes (CLI); Portainer: limited	No	Yes
Health monitoring	Basic status and metrics	Raw Docker stats / none	Metric graphs and alerts	Metrics + AI health status
Warns before a crash	No (alerts after failure)	No	Only if a fixed limit is crossed	Yes (Isolation Forest)
Needs labelled failure data	Not applicable	Not applicable	Not applicable	No (unsupervised)

2.3 Gap and Our Contribution

The gap is that deployment tools do not have any intelligence about container health, and fixed-threshold rules need constant tuning (they give false alarms in busy times and miss slow problems). OmniStack fills this gap by joining a simple deployment platform with an Isolation Forest health detector in one project. Our contribution is a working end-to-end system that is tested using injected faults, not only a model in a notebook.

2.4 How We Will Measure Success, Benchmarks and Competitive Approach

We will compare our model with a simple fixed-threshold rule (alert if CPU > 80% or RAM > 80%), which is how most basic monitoring works. Both will be tested on the same normal and fault-injected data. The numbers below are our targets and will be confirmed by testing.

What we measure	How we measure it	Target
Detection quality	Precision and recall on injected faults (memory leak, CPU spike, crash loop)	Recall ≥ 85%, higher than the threshold rule
False alarms	Wrong alerts on normal running containers	≤ 10% of normal readings
Early warning	How early the problem is flagged compared with the threshold rule	Earlier for memory leaks
Deployment	Time from GitHub link to running app on 3 sample apps	Under 10 minutes each

Table 2: Evaluation measures and targets

3. Project Methodology

We will work in two tracks at the same time: the platform track (Salman Khalid) and the AI/ML track (Talha Saleem). In the end we will join them. Before starting, we agree on a simple JSON format for metrics and results so both tracks can work separately.

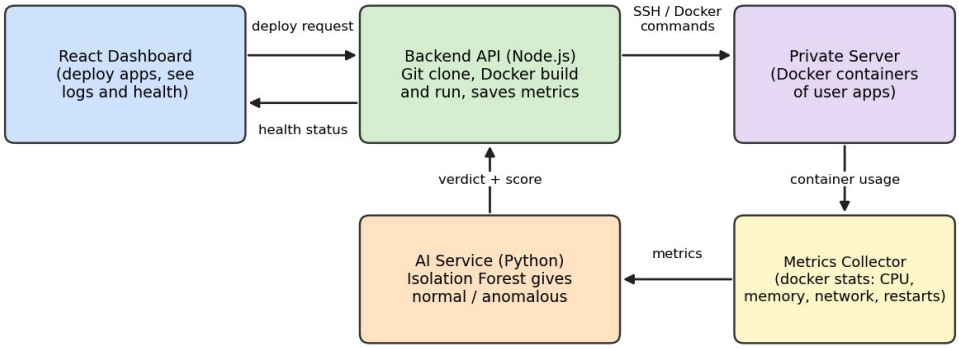


Figure 1: Block diagram of OmniStack

3.1 Components and Why We Need Them

- **React dashboard:** the user gives a GitHub link, starts the deployment and sees the app list and health status in one place.
- **Backend API (Node.js):** clones the repository, builds the Docker image and runs the container on the private server (over SSH or locally). It also saves the metrics and talks to the AI service.
- **Metrics collector:** reads CPU %, memory %, network I/O and restart count of each container using docker stats every few seconds. Network and restart values are converted to per-second and per-10-minute rates, because Docker only gives lifetime totals.
- **AI service (Python + scikit-learn):** holds the trained Isolation Forest model and returns a verdict (normal / anomalous) with a score for each container.

3.2 AI/ML Steps

1. Run sample apps (Node.js and Python) under normal load and record normal metrics.
1. Create abnormal data with fault scripts: memory leak, CPU spike and crash loop. This gives a test set with known answers.
2. Clean the data, scale the 4 features (CPU, memory, network rate, restart rate) and split into training and test sets.
3. Train Isolation Forest on normal data only, then tune its settings (number of trees, contamination) using the test set.
4. Measure precision, recall and false-alarm rate and compare with the threshold rule.
5. Wrap the model in a small API (FastAPI) so the backend can send metrics and get a verdict.

Tools: React, Node.js, Docker, Git/GitHub, Python, scikit-learn, FastAPI, Prometheus, Grafana, MS Project for planning. We work in weekly sprints and meet the advisor regularly.

4. Project Scope

Inside the scope (the system will provide)	Outside the scope (the system will not provide)
• Deploy a Node.js app from a GitHub link using Docker on a private server • Web dashboard with deployed apps, status and logs • Collection of CPU, memory, network I/O and restart count • Isolation Forest model that marks containers normal or anomalous • Fault-injection scripts for testing the model • Health status shown on the dashboard	• Many servers, Kubernetes or Docker Swarm • Auto-restart or self-healing of containers • AI chatbot and automatic log explanation (possible future work) • Automatic domain names and SSL setup (future work) • Cloud provisioning (AWS/GCP/Azure), user billing and multiple tenants • Redis for chaching (most probable future work) • Large-scale testing with many containers

5. High level Project Plan

The project is planned for 32 weeks (12 Oct 2026 to 21 May 2027) in MS Project. It shows the activities, the time given to each (in weeks), the resource who does it, and the university deadlines (proposal 13 Oct, SRS 10 Nov, design document 18 Dec, all before class).

ID	Task Name	Duration	Start	Finish	Res.
1	**Project Initiation & Learning**	**5 wks**	12 Oct 26	13 Nov 26	
2	Proposal document	1 wk	12 Oct 26	16 Oct 26	Both
3	Proposal deadline - 13 Oct (before class)	0 wks	13 Oct 26	13 Oct 26	
4	Learn Docker, Linux & Git basics	3 wks	19 Oct 26	06 Nov 26	Salman
5	Learn ML basics & Isolation Forest	3 wks	19 Oct 26	06 Nov 26	Talha
6	SRS document	3 wks	26 Oct 26	13 Nov 26	Both
7	SRS deadline - 10 Nov (before class)	0 wks	10 Nov 26	10 Nov 26	
8	**System Design**	**5 wks**	16 Nov 26	18 Dec 26	
9	Architecture & UI design	3 wks	16 Nov 26	04 Dec 26	Both
10	Database, API & model design	2 wks	30 Nov 26	11 Dec 26	Both
11	Design document	5 wks	16 Nov 26	18 Dec 26	Both
12	Design deadline - 18 Dec (before class)	0 wks	18 Dec 26	18 Dec 26	
13	**Platform Development**	**10 wks**	21 Dec 26	26 Feb 27	
14	Backend: clone repo & build Docker image	3 wks	21 Dec 26	08 Jan 27	Salman
15	Run containers on server & handle ports	3 wks	11 Jan 27	29 Jan 27	Salman
16	React dashboard (deploy & app list)	5 wks	18 Jan 27	19 Feb 27	Salman
17	Metrics collector (docker stats)	3 wks	08 Feb 27	26 Feb 27	Salman
18	**AI / ML Development**	**10 wks**	21 Dec 26	26 Feb 27	
19	Collect normal container metrics	3 wks	21 Dec 26	08 Jan 27	Talha
20	Fault injection scripts (leak, CPU, crash)	3 wks	11 Jan 27	29 Jan 27	Talha
21	Train Isolation Forest model	2 wks	01 Feb 27	12 Feb 27	Talha
22	Tune & evaluate the model	2 wks	15 Feb 27	26 Feb 27	Talha
23	Model API (FastAPI)	2 wks	15 Feb 27	26 Feb 27	Talha
24	**Integration**	**5 wks**	01 Mar 27	02 Apr 27	
25	Connect AI API to backend	3 wks	01 Mar 27	19 Mar 27	Both
26	Health status on dashboard & testing	4 wks	08 Mar 27	02 Apr 27	Both
27	**Testing & Evaluation**	**4 wks**	05 Apr 27	30 Apr 27	
28	Test with sample apps & fault injection	3 wks	05 Apr 27	23 Apr 27	Both
29	Compare with threshold rule & fix bugs	2 wks	19 Apr 27	30 Apr 27	Both
30	**Final Documentation & Defense**	**3 wks**	03 May 27	21 May 27	
31	Final report	2 wks	03 May 27	14 May 27	Both
32	Demo preparation & presentation	2 wks	10 May 27	21 May 27	Both
33	Final submission - 21 May	0 wks	21 May 27	21 May 27	

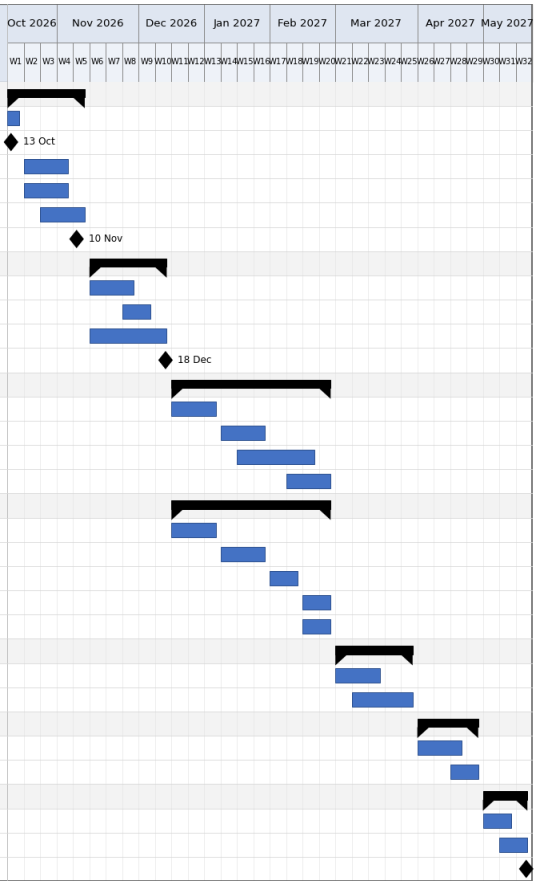


Figure 2: Project plan in MS Project (Gantt chart). Talha = AI/ML, Salman = DevOps

6. References

[1] Coolify, open-source self-hosted platform (our reference platform): https://coolify.io/docs and https://github.com/coollabsio/coolify

[2] Dokku docs: https://dokku.com/docs

[3] Prometheus Alertmanager docs: https://prometheus.io/docs/alerting/latest/alertmanager/

[4] YouTube: StatQuest with Josh Starmer (https://www.youtube.com/@statquest) and TechWorld with Nana (https://www.youtube.com/@TechWorldwithNana), used to understand Isolation Forest and Docker monitoring.

[5] AI conversations with Claude (Anthropic) and ChatGPT (OpenAI), Oct 2026, used for ideas on fault injection and project planning.

[6] Cloud With Varjosh: https://www.youtube.com/watch?v=wj8ELsJ-S5Q&t=40409s

Advisor consulted: Mr. Amir Zia

Project Supervisor's Signature: _____